

Docker Compose: servisi, mreže i volumeni

Praktikum: Razvojne operacije · Tjedan 4 · Blok A

Ishod bloka

Student zna objasniti kako iz koda nastaje slika i kada je treba ponovno izgraditi i objaviti, zašto više kontejnera opisujemo u compose.yaml, kako se servisi pronalaze na mreži, gdje podaci prežive kontejner i po čemu se razlikuju dugoživući i kratkoživući kontejneri.

Ponoviti	jednostavni Dockerfile i put od koda do slike u registru
Opisati	aplikaciju od više servisa jednom datotekom compose.yaml
Povezati	servise mrežom: host, zadani bridge, korisnička mreža i DNS
Sačuvati	podatke u bind mountu ili imenovanom volumenu
Razlikovati	poslužitelj (Ollama) i posao koji završi (pytest)

Sadržaj bloka

1	**Ponavljanje**	Jednostavni Dockerfile; od koda do slike u registru
2	**Docker Compose: kako i zašto**	compose.yaml, projekt i servisi, naredbe, redoslijed pokretanja
3	**Umrežavanje**	host, zadani bridge, korisnička mreža, DNS, objavljeni portovi
4	**Volumeni**	Sloj kontejnera, bind mount, imenovani volumen, životni vijek
5	**Dugoživući i kratkoživući**	Ollama poslužitelj vs pytest; restart, izlazni kod
6	**Provjera razumijevanja**	6 kratkih pitanja

1

Ponavljanje

Jednostavni Dockerfile i put od koda do registra

Jednostavni Dockerfile

Jedna faza, bez multi-stage builda: recept po kojem nastaje slika našeg repozitorija praktikum-<login>.

```
FROM python:3.13-alpine

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

COPY version.txt ollama_client.py llm.py ./

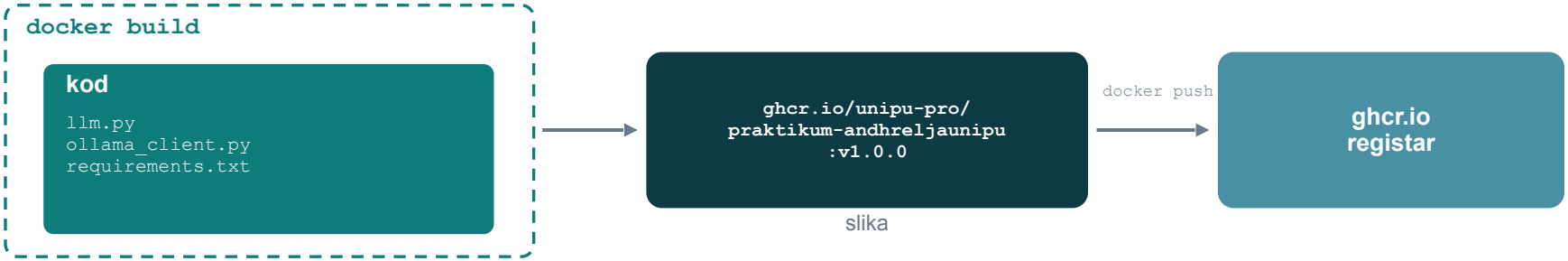
ENTRYPOINT ["python", "llm.py"]
CMD ["--help"]
```

1. **FROM**
   bazna slika: Alpine + Python 3.13 (prvi sloj)
2. **WORKDIR**
   sve sljedeće naredbe rade u /app
3. **COPY + RUN**
   prvo ovisnosti: sloj se gradi ponovno samo kad se requirements.txt promijeni
4. **COPY**
   kod na kraju: mijenja se najčešće, pa je cache iznad njega sačuvan
5. **ENTRYPOINT + CMD**
   što se pokreće: python llm.py, zadani argument --help

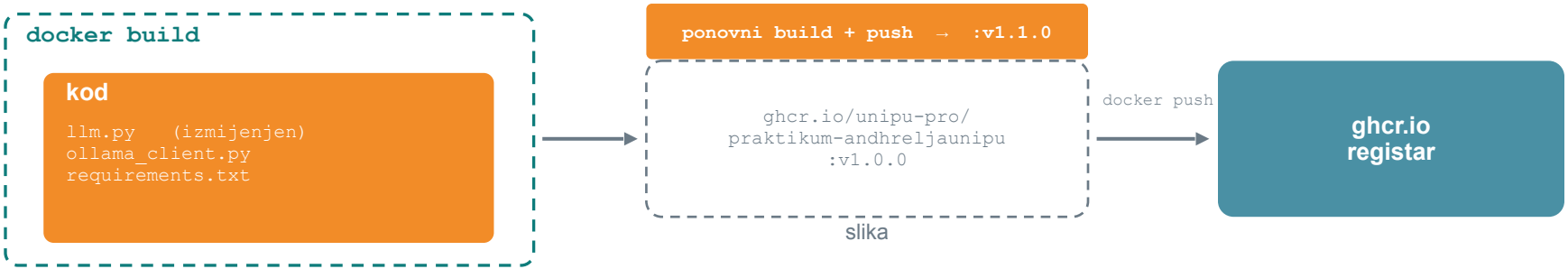
```
docker build -t praktikum-llm .        docker run --rm praktikum-llm status
```

Od koda do slike u registru

Faza 1 slika = kod u trenutku builda: registar poslužuje točno ono što je u gitu



Faza 2 kod je izmijenjen → v1.0.0 je zastarjela: ponovni build i push s novim tagom (v1.1.0)



Slika je snimka koda u trenutku builda i ne mijenja se sama. Svaka promjena koda znači novi build, novi tag (iz version.txt) i novi push; u tjednu 5 to radi GitHub Actions.

SHA-256: kod u slici je zamrznut

Svaki sloj i svaka slika imaju SHA-256 sažetak sadržaja. Promijeni li se ijedan bajt, mijenja se sažetak; isti sažetak znači isti sadržaj.

praktikum-andhreljaunipu:v1.0.0 image ID sha256:7f8def9e1a45		praktikum-andhreljaunipu:v1.1.0 image ID sha256:1df7be1fe9fb
COPY version.txt ollama_client.py llm.py sha256:f5736dbbf12f	≠	COPY version.txt ollama_client.py llm.py sha256:e3a870d88201
RUN adduser -D -u 10001 app sha256:1e8ff9656dbb	=	RUN adduser -D -u 10001 app sha256:1e8ff9656dbb
RUN pip install -r requirements.txt sha256:6f176d7e7c47	=	RUN pip install -r requirements.txt sha256:6f176d7e7c47
COPY requirements.txt . sha256:304a0a99fffa	=	COPY requirements.txt . sha256:304a0a99fffa
FROM python:3.13-alpine (7 slojeva) sha256:75c137aedfb2…	=	FROM python:3.13-alpine (7 slojeva) sha256:75c137aedfb2…

Primjer iz stvarnog builda (vaši sažeci bit će drugačiji). Promjena llm.py mijenja samo gornji sloj i ID slike; donji slojevi se dijele i ne preuzimaju ponovno.

Datoteka llm.py nakon izmjene

```
# host (IDE)
$ sha256sum llm.py
9d32cc8b5858…

# container from :v1.0.0
$ docker run --rm \
  --entrypoint sha256sum \
  …:v1.0.0 llm.py
7306749ee407…
```

Sažeci se razlikuju: kontejner i dalje izvodi kod iz trenutka builda.

Tag (:v1.0.0) je pokazivač koji se može premjestiti; digest (@sha256:…) uvijek znači točno isti sadržaj.

2

Docker Compose: kako i zašto

Više kontejnera, jedna datoteka, jedna naredba

Zašto Compose: od niza naredbi do datoteke

Bez Composea: naredbe koje treba zapamtiti

```
docker network create backend
docker volume create ollama-models
docker run -d --name ollama-server \
  --network backend \
  -v ollama-models:/root/.ollama \
  ollama/ollama:0.40.13
docker run -d --name open-webui \
  --network backend -p 3000:8080 \
  -e OLLAMA_BASE_URL=http://ollama-server:11434 \
  ghcr.io/open-webui/open-webui:v0.7.2
docker build -f Dockerfile.test -t tests .
docker run --rm --network backend tests
```

S Composeom: stanje opisano u gitu

```
docker compose up -d          # start the stack
docker compose run --rm tests # one-off job
docker compose logs -f        # follow logs
docker compose down           # tear down
```

Deklarativno

Opisujemo željeno stanje, Compose računa korake.

U gitu

compose.yaml se pregledava u PR-u kao i kod.

Ponovljivo

Isto okruženje kod svakog studenta i u CI-ju.

Compose ne zamjenjuje Docker: čita compose.yaml i šalje daemonu iste API pozive kao docker run, docker network create i docker volume create.

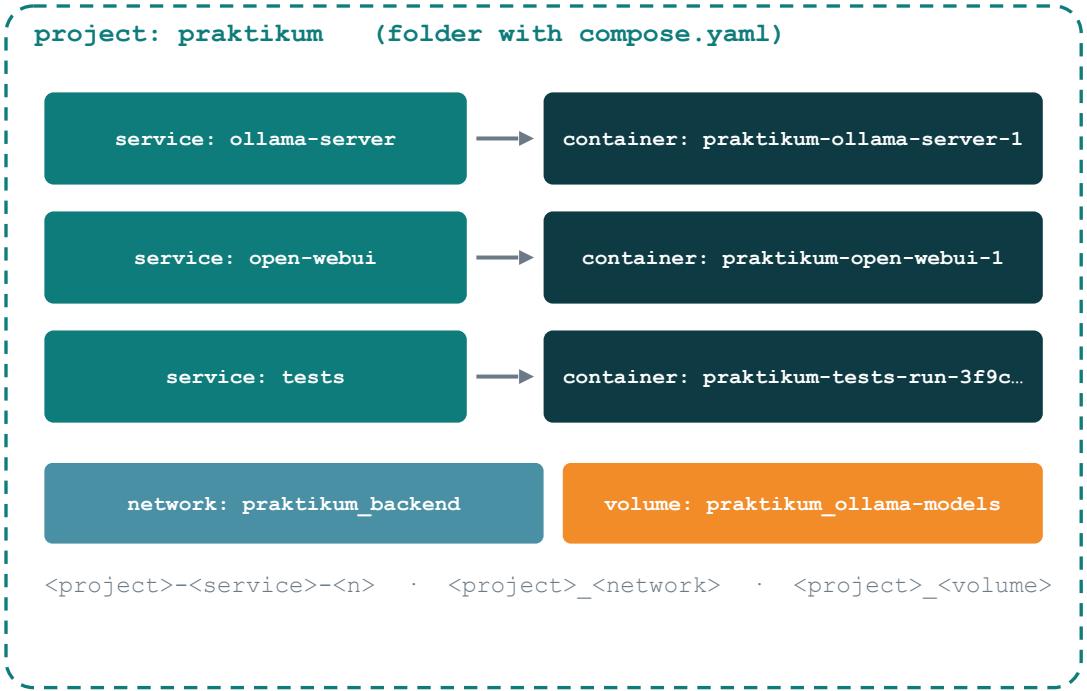
Anatomija compose.yaml

```yaml
name: praktikum
services:
  ollama-server:
    image: ollama/ollama:0.40.13
    networks:
      - backend
    volumes:
      - ollama-models:/root/.ollama
  open-webui:
    image: ghcr.io/open-webui/open-webui:v0.7.2
    networks:
      - backend
    ports:
      - "3000:8080"
    environment:
      OLLAMA_BASE_URL: http://ollama-server:11434
  tests:
    build:
      context: .
      dockerfile: Dockerfile.test
    networks:
      - backend
networks:
  backend:
volumes:
  ollama-models:
```

compose.yaml	docker naredba sa slajda 9
`name`	prefiks imena svih resursa
`services: <ime>`	`docker run -d --name <ime>`
`image`	slika na kraju `docker run`
`build`	`docker build -f Dockerfile.test`
`networks: (servis)`	`--network backend`
`volumes: (servis)`	`-v izvor:odredište`
`ports`	`-p 3000:8080`
`environment`	`-e OLLAMA_BASE_URL=...`
`networks: (dno)`	`docker network create backend`
`volumes: (dno)`	`docker volume create ollama-models`

depends_on, healthcheck, restart i profiles: sljedeći slajdovi i vježba.

Projekt, servis, kontejner



Servis ≠ kontejner

Servis je opis u compose.yaml; kontejner je pokrenuti primjerak. Jedan servis može imati više kontejnera.

Ime projekta

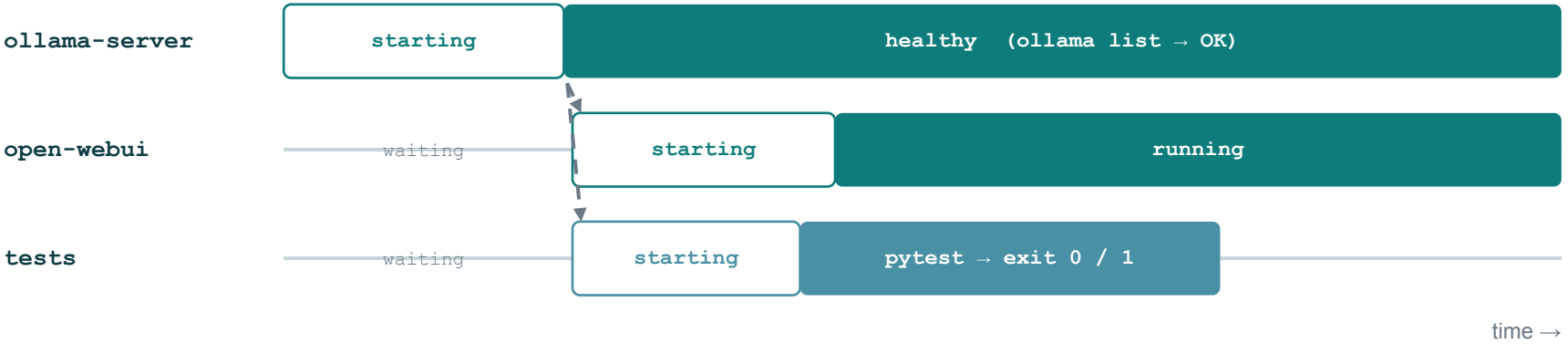
name: u datoteci, -p ili ime direktorija. Dva projekta s istim imenom dijele resurse.

Osnovne naredbe

Naredba	Primjer	Što radi
`up`	`docker compose up -d`	Gradi/preuzima slike, stvara mreže i volumene, pokreće servise u pozadini
`ps`	`docker compose ps -a`	Kontejneri projekta; -a i završeni (Exited)
`logs`	`docker compose logs -f ollama-server`	Ispis jednog ili svih servisa; -f prati
`exec`	`docker compose exec ollama-server ollama list`	Novi proces u kontejneru koji već radi
`run`	`docker compose run --rm tests`	Novi, jednokratni kontejner servisa; vraća njegov izlazni kod
`build`	`docker compose build`	Gradi slike servisa koji imaju build:
`config`	`docker compose config`	Konačna konfiguracija: varijable uvrštene, greške u YAML-u
`stop`	`docker compose stop`	Zaustavlja kontejnere (SIGTERM, pa SIGKILL), ne briše ih
`down`	`docker compose down`	Briše kontejnere i mreže; volumeni ostaju
`down -v`	`docker compose down -v`	Briše i imenovane volumene (modeli se preuzimaju ponovno)

exec ulazi u postojeći kontejner; run stvara novi. up je za poslužitelje, run --rm za poslove koji završe.

Redoslijed: depends_on i healthcheck



condition	Compose čeka da ovisnost...	Primjer
`service_started`	bude pokrenuta (zadano; ne znači da je spremna)	depends_on: [ollama-server]
`service_healthy`	prođe healthcheck	open-webui i tests čekaju ollama-server
`service_completed_successfully`	završi s izlaznim kodom 0	jednokratni posao prije servisa

3

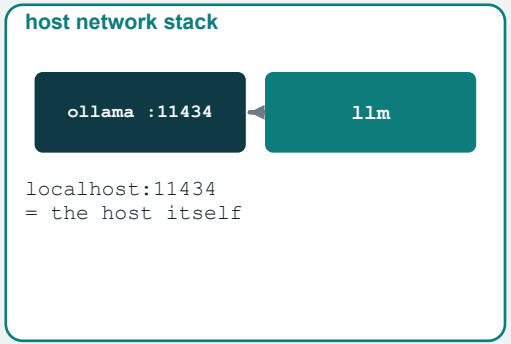
Umrežavanje

host, zadani bridge, korisnička mreža i DNS

Tri mrežna načina

host

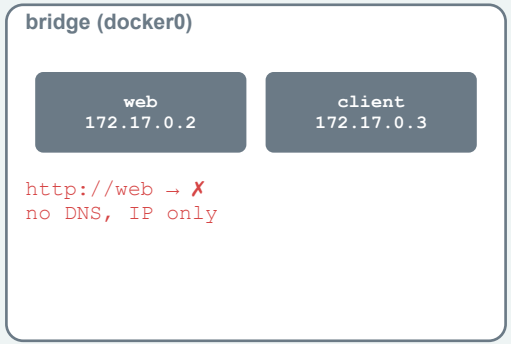
`network_mode: host`



Kontejner dijeli mrežu hosta: nema izolacije ni ports:. Blok B: alat do Ollame na hostu.

default bridge

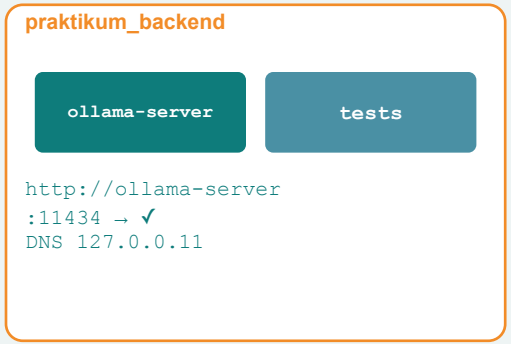
`network_mode: bridge`



Najstarija Docker mreža. Nema DNS-a po imenu; Compose je ne koristi bez network_mode: bridge.

user-defined bridge

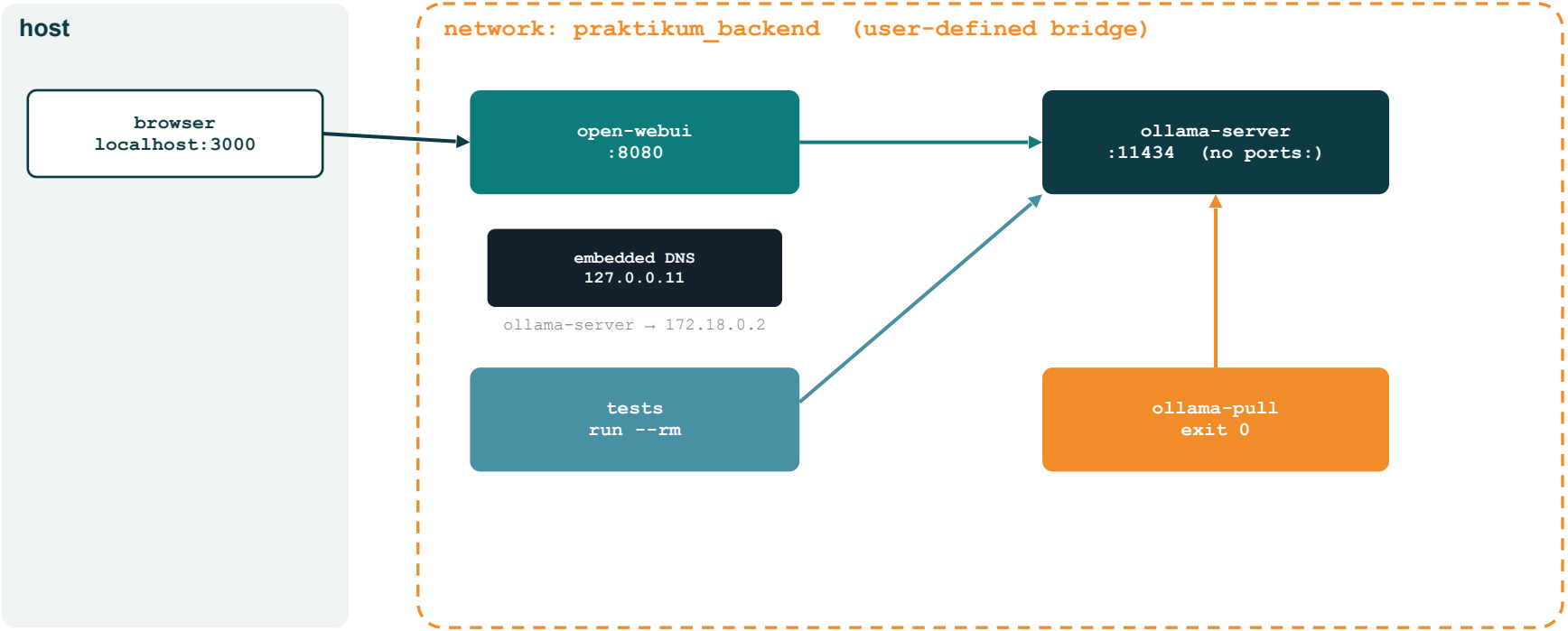
`networks: [backend]`



Compose je stvara sam (<projekt>_default) ili po imenu. Ugrađeni DNS: ime servisa → IP.

Pravilo: servisi istog projekta zovu se http://<servis>:<port>; IP adrese se mijenjaju, imena ostaju.

Mreža projekta, DNS i objavljeni portovi



Preglednik vidi samo objavljeni port (open-webui). Ollama nema ports: i dostupna je samo kontejnerima na mreži backend.

Objavljivanje portova i DevSecOps

U compose.yaml	Tko može doći do servisa	Kada
`ports: ["3000:8080"]`	svi koji vide računalo (0.0.0.0): i kolege na Wi-Fi mreži	rijetko; javni servis iza proxyja (tema 13)
`ports: ["127.0.0.1:3000:8080"]`	samo ovo računalo	lokalni razvoj (open-webui)
`bez ports:`	samo kontejneri na istoj mreži	baze, Ollama, interni API-ji
`networks: {backend: {internal: true}}`	ni kontejneri na toj mreži ne vide internet	pozadinski servisi bez izlaza van

Najmanja izloženost

Objaviti samo ono što korisnik koristi. Svaki objavljeni port je meta.

Lijevo host, desno kontejner

3000:8080 = port 3000 na hostu vodi na 8080 u kontejneru.

Autentikacija

WEBUI_AUTH=False samo lokalno. Na zajedničkom VM-u (tema 6) uvijek prijava.

Host mreža: kada da, kada ne

Prednosti

localhost u kontejneru je host: vidi Ollamu koja radi na računalu.
Nema mapiranja portova.
Brz prijelaz s lokalnog razvoja na kontejner (blok B).

Nedostaci

Nema mrežne izolacije: kontejner vidi sve portove hosta.
Sukobi portova s hostom; nema DNS-a po imenu servisa.
Ne kombinira se s ports: ni networks:.

Okruženje	network_mode: host
Docker Engine (Linux, WSL)	radi bez postavki
Docker Desktop 4.34+ (Windows, macOS)	Settings → Resources → Network → Enable host networking
Starije verzije Desktopa	ne radi; zamjena: host.docker.internal

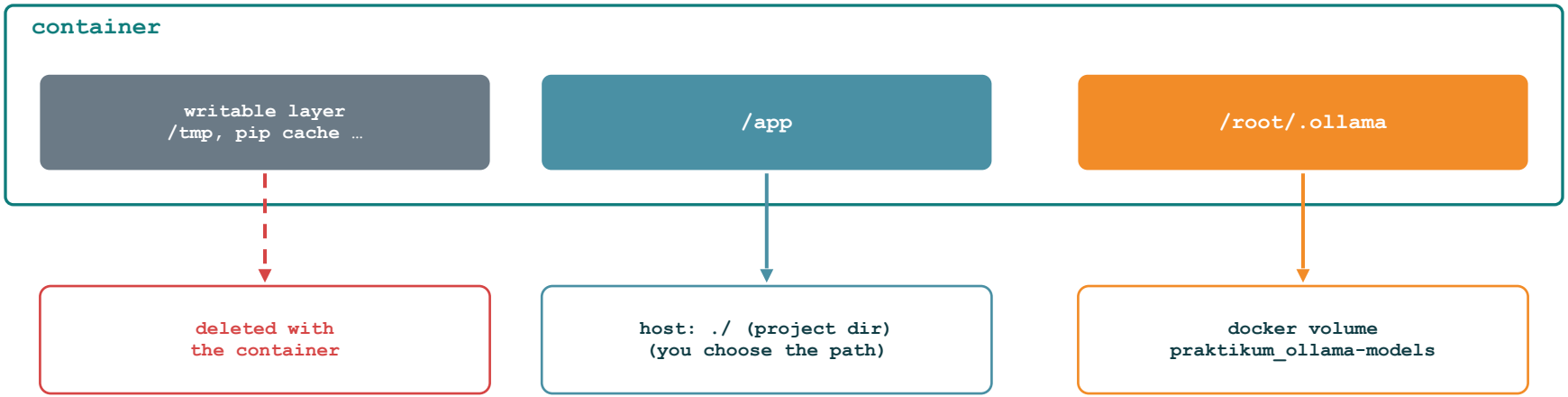
U vježbi: blok B koristi host mrežu, blok C seli Ollamu u Compose i prelazi na mrežu backend.

4

Volumeni

Gdje podaci prežive kontejner

Gdje žive podaci kontejnera



Sloj kontejnera

Nestaje s docker rm / --rm. Za privremene datoteke.

Bind mount

./:/app. Direktorij hosta koji biramo sami; izmjena u editoru odmah je u kontejneru.

Imenovani volumen

ollama-models:/root/.ollama. Docker upravlja mjestom; prenosiv, ne ovisi o putanji.

Postoji i tmpfs (samo u memoriji, za tajne i privremene podatke); ovdje ga ne koristimo.

Volumeni u compose.yaml i njihov životni vijek

```yaml
services:
  ollama-server:
    volumes:
      - ollama-models:/root/.ollama    # named
  llm:
    volumes:
      - ./:/app                        # bind
      - ./data:/app/data:ro            # read-only

volumes:
  ollama-models:                      # declared here
```

Ime ispred : = volumen; putanja (./, /) = bind mount.

Naredba	Sloj	Bind	Imenovani
`docker compose stop`	ostaje	ostaje	ostaje
`docker compose down`	briše	ostaje	ostaje
`docker compose down -v`	briše	ostaje	briše
`docker volume prune`	–	ostaje	briše nekorištene

Vlasnik datoteka u bind mountu

Kontejner piše kao svoj korisnik (često root, uid 0). Na Linuxu i u WSL-u datoteke na hostu tada pripadaju rootu: --user "$(id -u):$(id -g)".

Podaci koje želimo zadržati (modeli, baza, korisnički podaci) nikad ne ostaju u sloju kontejnera.

5

Dugoživući i kratkoživući

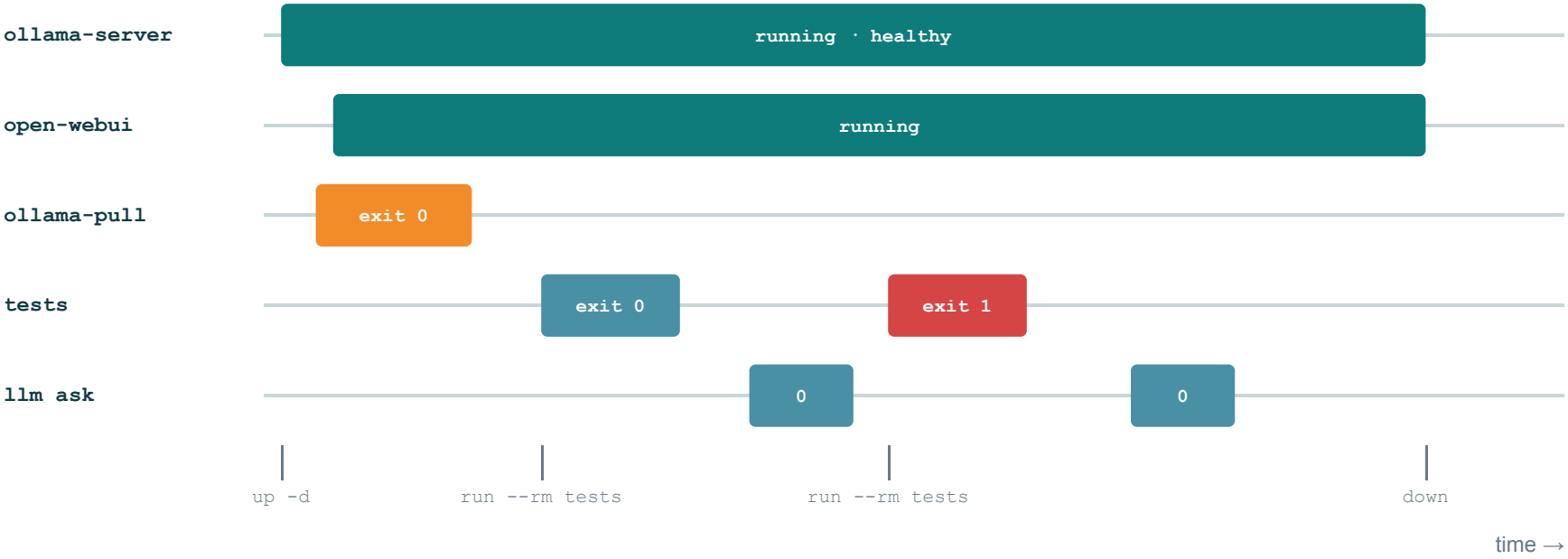
Poslužitelj koji radi i posao koji završi

Dugoživući i kratkoživući kontejneri

	Dugoživući: ollama-server	Kratkoživući: tests (pytest)
Svrha	poslužuje zahtjeve dok ga ne zaustavimo	obavi jedan posao i izađe
Pokretanje	docker compose up -d	docker compose run --rm tests
Rezultat	odgovori na HTTP zahtjeve	izlazni kod (0 = uspjeh, ≠ 0 = pad)
Zdravlje	healthcheck: ollama list	nema; trajanje je ograničeno
restart	unless-stopped	"no"
Ovisnost	nema; on je backend	treba backend: depends_on (blok C)
Gašenje	SIGTERM pri stop/down; mora se ugasiti uredno	završi sam; --rm briše kontejner
Primjeri	baze, API-ji, Open WebUI	testovi, migracije, ollama-pull, llm ask

Isti alat može biti oboje: llm chat je interaktivni dugoživući proces, llm ask jednokratni posao.

Životni vijek kontejnera kroz jedan dan vježbe



Izlazni kod kratkoživućeg kontejnera je njegov rezultat: na njemu CI u tjednu 5 odlučuje je li build zelen.

restart i izlazni kod

restart	Ponovno pokreće kontejner...
`"no" (zadano)`	nikad
`on-failure[:3]`	samo kad izađe s kodom ≠ 0 (najviše 3 puta)
`always`	uvijek, i nakon docker stop kad se daemon restarta
`unless-stopped`	uvijek, osim ako ga nismo ručno zaustavili

Kod	Značenje
`0`	uspjeh
`1`	greška (npr. pao test)
`137`	128 + 9: SIGKILL
`143`	128 + 15: SIGTERM

Zašto ne restart: always za testove?

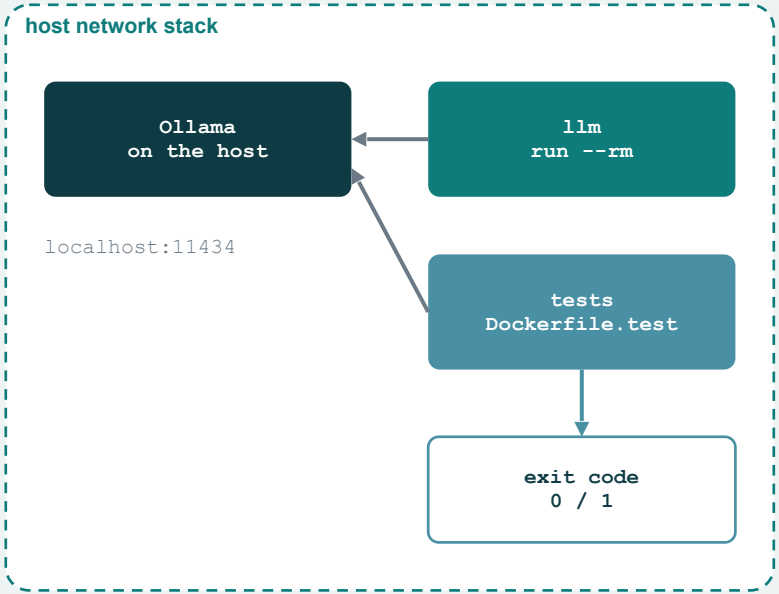
Testovi bi se vrtjeli u krug: završe, Docker ih ponovno pokrene. Jednokratni poslovi imaju "no".

Kako tests čeka backend?

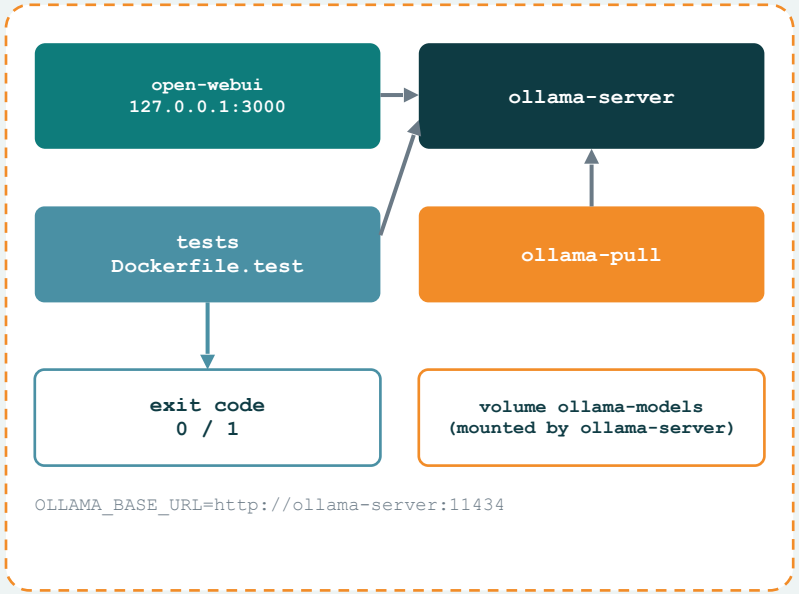
depends_on: ollama-server: condition: service_healthy. Compose pokreće pytest tek kad Ollama odgovara na healthcheck.

Što gradimo u vježbi

Block B: host network



Block C: network backend



Isti testovi i ista slika; mijenja se samo OLLAMA_BASE_URL. Konfiguracija dolazi iz okoline, ne iz slike.

Provjera razumijevanja

1. Promijenili ste llm.py. Zašto ghcr.io/…/praktikum-andhreljaunipu:v1.0.0 i dalje sadrži stari kod i što treba napraviti?
2. Koja je razlika između docker compose run --rm tests i docker compose exec ollama-server ollama list?
3. Servis tests ima network_mode: bridge i ne može doći do http://ollama-server:11434. Zašto?
4. Što od podataka ostaje nakon docker compose down, a što nakon docker compose down -v?
5. Zašto ollama-server nema ports:, a open-webui ima 127.0.0.1:3000:8080?
6. Kako Compose osigurava da integracijski testovi ne krenu prije nego što Ollama odgovara?